

GRAMMAR CORRECTION AND NEXT WORD PREDICTION USING LONG SHORT - TERM MEMORY AND GATED RECURRENT UNIT

A Dissertation submitted in partial fulfillment
of the requirements for the degree
of

INTEGRATED MASTER OF SCIENCE
IN
DATA SCIENCE

by

MADHINI S
CB.PS.I5DAS22072



DEPARTMENT OF MATHEMATICS
AMRITA SCHOOL OF PHYSICAL SCIENCES
AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112 (INDIA)

April 2026

AMRITA SCHOOL OF PHYSICAL SCIENCES

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112



AMRITA
VISHWA VIDYAPEETHAM

BONAFIDE CERTIFICATE

This is to certify that the dissertation entitled “**Grammar correction and next word prediction using LongShort-Term Memory and Gated Recurrent Unit**” submitted by **MADHINI S (Reg. No: CB.PS.I5DAS22072)**, for the award of the **Degree of Integrated Master of Science in Data Science** is a bonafide record of the work carried out by her at the Department of Mathematics, Amrita School of Physical Sciences, Amrita Vishwa Vidyapeetham, Coimbatore.

Signature of Examiner

Signature of Project Coordinator

Signature of Chairperson

AMRITA SCHOOL OF PHYSICAL SCIENCES

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

DEPARTMENT OF MATHEMATICS

DECLARATION

I, Ms. Madhini S (Reg. No: CB.PS.I5DAS22072), hereby declare that this dissertation entitled “Grammar correction and next word prediction using LongShort-Term Memory and Gated Recurrent Unit”, is the record of the original work done by me at Department of Mathematics, Amrita School of Physical Sciences, Amrita Vishwa Vidyapeetham, Coimbatore. To the best of my knowledge this work has not formed the basis for the award of any degree/diploma/ associateship/fellowship/or a similar award to any candidate in any University.

Place: Coimbatore

Signature of the Student

Date:

COUNTERSIGNED

Dr. Sangeeta Kumari

Project Coordinator

Department of Mathematics

Contents

Acknowledgement	iii
List of Symbols	iv
List of Figures	v
Abstract	1
1 Introduction	2
Introduction	2
2 Literature Review	3
Literature Review	3
3 Problem Statement	5
4 Objectives	6
5 System Overview	7
5.1 System Architecture	7
6 Proposed Model	9
6.1 Data Collection	9
6.2 Data Preprocessing	9
6.3 Grammar correction	10
6.4 Model Development	10
6.4.1 LSTM Algorithm	10
6.4.2 GRU Algorithm	11
6.5 Training Process	11
6.6 Prediction	11
6.7 Model Selection	12
7 Model Architecture	13
7.1 LSTM Architecture	13
7.1.1 Forget Gate	13
7.1.2 Output Gate	13

7.2	GRU Architecture	14
7.2.1	Update Gate	14
7.2.2	Reset Gate	14
8	Results and Analysis.....	15
8.1	Accuracy Comparison	15
8.2	Confidence Comparison	15
8.3	Observations	16
9	Conclusion	17

Acknowledgement

Amrita Vishwa Vidyapeetham, Coimbatore, for the continuous encouragement, constructive feedback, and patient guidance extended throughout each stage of this work. Their thoughtful direction helped transform an initially loosely defined set of ideas into a coherent and functional system.

My sincere thanks to Dr. K. Somasundaram, Chairperson, Department of Mathematics, Amrita School of Physical Sciences, for his encouragement and for ensuring access to the necessary departmental facilities.

I am deeply grateful to my project Advisor Dr. Sangeeta Kumari, Department of Mathematics, Amrita School of Physical science, for their guidance, encouragement, criticism, and faith. They have been a role model for me, with their broad knowledge, inquisitive mind, uncompromising integrity, and enviable ability in carrying out my project works. Their dedication and support has inspired and has left an indelible mark in my memory.

I acknowledge my deep sense of gratitude and dedicate this work to my family members and friends who have always been a source of inspiration to me throughout my study. I am so grateful for their patience, love and care they have shown during the period of my project work.

(Madhini S)

List of Abbreviations

NLP	Natural Language Processing
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
RNN	Recurrent Neural Network
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
API	Application Programming Interface
CPU	Central Processing Unit

List of Figures

5.1	Architecture of the proposed grammar correction and next-word prediction system	7
6.1	Proposed system architecture combining grammar correction, dual-model (LSTM and GRU) prediction, and confidence-based selection mechanism	12
7.1	Architecture of Long Short-Term Memory (LSTM) network showing forget gate, input gate, output gate, and memory cell operations	13
7.2	Architecture of Gated Recurrent Unit (GRU) showing reset gate, update gate, and hidden state computation	14
8.1	Top-3 accuracy comparison between LSTM and GRU models	15
8.2	Comparison of LSTM and GRU models based on accuracy and average confidence scores . .	16

Abstract

Natural Language Processing is a part of Artificial Intelligence. It helps machines understand what people are saying and writing. Because we are using computers and phones to talk to each other more we need tools that can help us write correctly and guess what we want to say next. Natural Language Processing does things like fix our grammar mistakes. Predict the next word we will type. These things are very useful. Make it easier for us to use computers and phones. Natural Language Processing is used for things but two of the most important things it does are grammar correction and next word prediction. Natural Language Processing makes a difference, in how we use tools and platforms. We use Natural Language Processing when we want to make sure our sentences are grammatically correct. Natural Language Processing is also used to predict the word we will type which saves us time. This project is about designing and building a system that corrects grammar and predicts the word. It uses deep learning techniques. The system has two parts: One part corrects grammar using rules, The other part predicts the next word using networks The grammar correction part finds and fixes common mistakes. These include verb use and subject-verb agreement errors. This step cleans up the text before it goes to the prediction models. The system uses Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models. These models help predict the word. The goal is to make the system accurate and reliable. It can be used to improve writing and communication. The system combines rule-based and sequence-based models. This makes it effective, in correcting grammar and predicting words. The project looked at how the LSTM and GRU models work. Used things like how the models got the top 3 answers right and how confident they were in their answers. What they found out is that the GRU model is better than the LSTM model. It is better at getting the answers and it is also faster. In conclusion, this project successfully achieves its objective of developing an intelligent system for grammar correction and next-word prediction. The results suggest that GRU is a more efficient and reliable model for this application, and the system can be further enhanced by incorporating advanced techniques such as transformer-based models and larger datasets in future work.

Chapter 1

Introduction

Natural Language Processing is an area in Artificial Intelligence. It helps computers understand language. Natural Language Processing enables computers to make sense of how people communicate. This is crucial with all the communication we do through emails and messaging apps. The need for tools to process text has increased a lot lately. This is because we use tools, like documentation software more and more. Two important uses of Natural Language Processing are grammar correction and word prediction. These tools help improve how we write and communicate online. Natural Language Processing makes it possible to correct grammar mistakes. It also helps predict what word comes next when we type. Natural Language Processing and grammar correction are closely related. Natural Language Processing and word prediction are applications. Grammar correction systems help fix mistakes in writing. These mistakes include verb tenses, incorrect use of subjects and verbs and sentences that are not structured well. The system helps users write sentences that're grammatically correct. In contrast next word prediction systems help users type faster. They do this by suggesting the word based on what the user has typed so far. This feature uses the context to make predictions. It helps users type faster and more accurately. Traditional methods for fixing grammar mistakes and predicting text have mainly used rule-based systems. These systems are easy to set up. They do not work well with complicated sentences and everyday language. The problem is, they can't grasp the situation and struggle with information. Grammar correction and text prediction need to handle sentences. They have to understand context to work properly. Rule-based systems are not effective in dealing with inputs. They are not efficient in dealing with sentences. Traditional approaches, to grammar correction rely on these systems. They are not able to understand language usage. Deep learning is getting better and better. Now we have models like Long Short-Term Memory and Gated Recurrent Unit that're really good at handling sequences. These Long Short-Term Memory and Gated Recurrent Unit models can look at a lot of data. Find relationships between words.

Long Short-Term Memory is special because it has memory cells and gates. This means Long Short-Term Memory can remember things that happened a time ago. Gated Recurrent Unit is similar, to Long Short-Term Memory. It works faster and uses fewer parameters. However, with the development of deep learning techniques, models like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) have gained popularity in sequence-based models. These models have the ability to learn and recognize data from large data sets and understand the context between words. The project aims to develop a grammar correction and next word prediction using models like LSTM and GRU. The project will correct the grammar of the given text using rule-based approaches and then use the trained models of the neural network to predict the next word in the text. A comparative analysis will be done to compare the performance of the models like LSTM and GRU on the basis of accuracy and confidence.

Chapter 2

Literature Review

Over the last several decades, there have been considerable developments in the domain of Natural Language Processing (NLP). There have been efforts to develop machines that can process natural languages efficiently. In early attempts at developing solutions to problems such as grammatical error correction and next word prediction, the rule-based method was the prevalent approach used. In the rule-based approach, rules for grammar were hand-crafted based on the knowledge of linguists and other experts in various domains. Rule-based methods can be applied to relatively straightforward sentences and perform well when it comes to simple grammatical errors, such as subject-verb disagreement or incorrect tenses. But their application is restricted because it is not easy to write rules that account for all possible cases of a diverse and ambiguous natural language.

In order to solve the above-stated problems, there appeared some methods that employed statistical analysis to deal with language processing. In order to achieve this purpose, some probabilistic approaches were developed that allowed modeling a language based on large collections of texts. The most popular example of this approach is the n-gram model when the probability of a word was computed on the basis of the preceding $n-1$ words. It turned out that n-gram models performed well in cases of next-word prediction and text generation for purposes of search engines or primitive text processing systems. Nevertheless, there were several drawbacks of these methods, one of which was data sparseness, as well as the inability to take into account the dependence on distant words.

The advent of deep learning was a revolutionary development in the field of NLP since deep neural networks could now learn sophisticated structures from raw data without the need to engineer handcrafted features. One of the first applications of deep learning was to develop Recurrent Neural Networks (RNNs), which were used to solve sequential problems in natural language processing. An RNN maintains a hidden state, which gets updated through time, giving it the ability to remember the input sequence so far. The application of RNNs in NLP has allowed researchers to solve a range of sequential problems such as language modeling and word prediction problems. However, one of the major challenges faced by RNNs is that of vanishing gradients.

As a solution to this problem, LSTM neural networks were introduced as an improvement on the RNN approach. The LSTM network employs the use of memory cells coupled with three gates: the input gate, the forget gate, and the output gate. This design allows the neural network to control what information is retained and what is not, thus allowing the model to capture the dependencies between data points. It is due to this characteristic that LSTM models have been used extensively in NLP applications, such as language modeling, speech recognition, and machine translation.

After LSTM, came another simpler version called Gated Recurrent Unit (GRU). GRU makes use of some features of the LSTM network by making some modifications in the LSTM architecture. In particular, GRU makes use of only two gates: the update gate and the reset gate. This makes GRU more computationally efficient than LSTM because it is easier to train the GRU. There have been many research papers indicating that GRUs perform equally as well as LSTM in many natural language processing tasks. Also, GRU becomes advantageous for real-time processes due to the reduced computational time involved.

Hybrid methods involving the combination of rules and deep learning models have been researched by

scientists in recent times. In such hybrid models, grammatical corrections of the text are made at the preprocessing stage when the text is fed to neural networks. It makes input data of higher quality which helps to enhance the predictive accuracy. These solutions make use of the advantages of both rule-based systems and deep learning models.

In this project, LSTM and GRU models will be developed for performing the task of next word prediction. The performance of these two models will be compared. The research in this field shows that while LSTM can successfully handle long-term dependencies, the GRU model works better due to its faster performance. Therefore, choosing the optimal architecture depends on the requirements of each specific case.

Chapter 3

Problem Statement

In today's world we use text to communicate a lot. People often chat on messaging apps send emails and use platforms to talk to each other. Sometimes we make mistakes when typing, like grammar errors. This happens because we are in a hurry not good at the language or just typing quickly. One big problem is fixing these grammar mistakes automatically. Old grammar tools use man-made rules. They are not good at handling tricky sentences or what the sentence really means. They get confused with sentences that're unclear or that they have not seen before. Another issue is guessing the word in a sentence. This requires understanding how the words fit together and what the sentence is about. Guessing the word is not easy because language is always changing and depends on the situation. The model needs to understand how words relate to each other even if they are apart in the sentence. Some key challenges with the grammar correction project are:

- Understanding the sentence: The system needs to get what the sentence means to guess the word
- Handling word order: Words in a sentence depend on each other. Keeping track of this is hard for models.
- Grammar ambiguity: Some sentences can be correct in ways, which makes fixing them tricky.
- Rule-based systems: Rule-based systems cannot handle all kinds of sentences.
- Model efficiency: The system should make predictions that work for real-time applications.

To solve these problems the grammar correction project uses learning models like Long Short Term Memory and Gated Recurrent Unit. These models are good at handling data and learning how words relate to each other. So the main problem the grammar correction project tries to solve is: to make a system that can automatically fix grammar mistakes and guess the word, in a sentence using learning and to compare how well Long Short Term Memory and Gated Recurrent Unit models work.

Chapter 4

Objectives

The main purpose of this project is to create an intelligent system that is able to correct the grammar as well as predict the next word using modern deep learning methods. The system has been designed in such a way that it would improve the grammatical structure of the text entered into it and predict words which would be suitable according to the context. Such a combination will make this system useful in developing applications like smart typists, chatbots, etc.

In order to realize this goal, the major focus will be laid on creating an effective grammar correcting mechanism in the system that would be able to correct grammatical mistakes occurring in the sentences. These mistakes will mainly be in terms of incorrect verb tenses, wrong subject-verb agreements, and other elementary sentence structures. Using rule-based grammar correction techniques will ensure the grammatical soundness of the text entered.

Apart from grammatical corrections, another important aspect that this project focuses on is the application of deep learning models that deal well with sequential data. The language data itself is sequential because the meaning of a particular word is dependent on the words that have come before it in a sentence. Two types of neural networks that help in learning the contextual relationship between different data points are employed: Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU).

One of the aims of the project is to be able to predict the next word in a sentence. In other words, the machine should learn from the words that it receives as inputs and predict the next most likely word. The predicted word will not only be grammatically correct but also have context in relation to the sentence given to the model.

The performance evaluation strategy of the system is presented as well. The performances of LSTM and GRU models are assessed through metrics including Top-3 Accuracy and Prediction Confidence. The first metric measures whether the right word is included in the list of the next-word predictions within the first three places – which is especially important in practical applications such as text auto-completion. The second measure is used to evaluate how likely the model considers its predictions, i.e., what is the probability given to each prediction.

Also, a comparative analysis of the two types of models is performed. The comparison will be made on the basis of such aspects as prediction accuracy, training process efficiency, and general effectiveness when working with sequences.

In conclusion, the current project attempts to combine grammar correction and text prediction features in order to develop an effective text-generation system.

Chapter 5

System Overview

The system has been built as a pipeline with multiple stages for transforming the input text into meaningful predictions. The workflow starts with the user input, where the user is asked to enter the sentence. The sentence may or may not have grammatical errors. The sentence is then passed through the grammar correction module, where the grammatical errors are corrected. The sentence is then preprocessed, which involves tokenization and conversion into numerical form. The data is then passed through the LSTM and GRU models. The final prediction for the next word is made, along with the confidence level. The final prediction is made by comparing the outputs of the two models. The advantages of the proposed modular design are:

- Scalability
- Flexibility
- Easy integration with the application

5.1 System Architecture

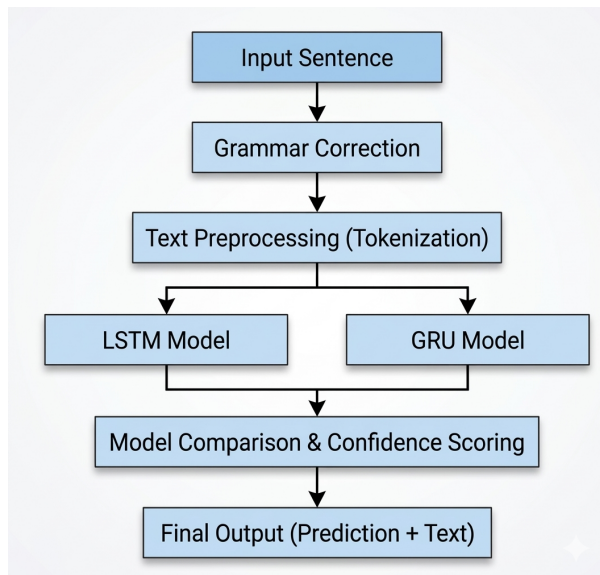


Figure 5.1: Architecture of the proposed grammar correction and next-word prediction system

The system architecture represents the workflow of the proposed model. The input sentence is first processed through a grammar correction module to fix common errors. The corrected text is then preprocessed using tokenization techniques. The processed input is fed into both LSTM and GRU models for next-word prediction. Each model generates predictions along with confidence scores. A model comparison module

evaluates both outputs and selects the best prediction. Finally, the system outputs the corrected sentence along with the predicted next word.

Chapter 6

Proposed Model

The methodology explains the process followed in the development of the system.

6.1 Data Collection

Data gathering is the first and foremost process in constructing the language model. Quality and richness of the data will have an impact on the accuracy of the model built.

In the current project, data gathered are textual sentences extracted from literary works and general English corpus. The variety of structures, vocabularies, and styles in sentences will provide the context learning by the model in order to make predictions.

Sources of data are as follows:

- Literary works (e.g., Shakespeare dataset)
- General English sentences
- Open source text corpus
- Textual data is kept as plain text files, which are the inputs of further processing.

6.2 Data Preprocessing

This process is used to preprocess raw textual data so as to make it structured and consistent for deep learning model training.

Steps involved in the process include:

- Lowercasing

In order to maintain consistency, all text data is made lowercase to ensure that one does not treat the same word as different tokens.

Example: "Hello World" → "hello world"

- Removal of punctuation

Punctuation marks like periods, commas, and other symbols are eliminated since they do not help much in predicting the next word.

Example: "Hello, world!" → "hello world"

- Tokenization

Tokenization is the breaking down of sentences into words or tokens. Here, the sentences will be broken down into individual words.

Example: "she goes home" → ["she", "goes", "home"]

- Creation of vocabulary

In this step, a vocabulary is created where each word is assigned a unique index number. It will make it possible for processing textual data numerically.

Example:

Word Index she 1 goes 2 home 3

The vocabulary size depends on the dataset in use.

6.3 Grammar correction

Grammar correction is added to make sure that the generated words are not only contextually related but also grammatically accurate. Grammar correction is done using rule-based algorithms, which emphasize grammatical rules in their simplest form.

Subject-Verb Agreement The algorithm makes sure that the verb is properly formed according to the subject's person and number. Examples: "she go" → "she goes" "they goes" → "they go"

Sentence Structure The algorithm verifies whether the sentence has proper grammatical structure, such as:

- Subject + Verb + Object
- Correct usage of verb tenses
- Prediction errors are either removed or corrected according to set rules.
- Significance of Grammar Correction
- prediction accuracy
- Increases readability
- Boosts the effectiveness of the application

6.4 Model Development

The model implements two types of deep learning algorithms: LSTM and GRU.

6.4.1 LSTM Algorithm

LSTM stands for Long Short-Term Memory; it is an algorithm belonging to the category of recurrent neural networks.

Characteristics of the algorithm:

- Memory blocks for storing previous data
- Input, forget, and output gates
- Effective work with lengthy sequences

Advantages of the algorithm:

- Improved context recognition
- Appropriate for language modeling

6.4.2 GRU Algorithm

Characteristics of the algorithm:

- Update gate
- Reset gate
- Quicker training process than LSTM

Advantages of the algorithm:

- Lower computational complexity
- Training process is faster than LSTM
- Performance comparable to LSTM in certain cases

6.5 Training Process

The training procedure consists of feeding the sequences to the model and adjusting their weights according to the prediction errors.

Training Steps

- Generation of input sequences from the data set
- Predicting the next word
- Comparing the predicted word with the actual word
- Calculating loss through loss functions such as categorical cross-entropy
- Adjusting the weight of the model using backpropagation
- Repeating the training procedure for several epochs
- Backpropagation adjusts the weight of the model to reduce the value of the loss function.
- One epoch equals one pass of the data set through the network Several epochs are conducted to enhance the accuracy of the model

6.6 Prediction

Once the model has been trained, it is utilized to predict the next word from the input sequence.

Word Prediction Steps

- The user enters a sentence

- The sentence undergoes preprocessing and tokenization
- The trained model generates probabilities for the next possible word
- The word with the maximum probability is chosen Confidence Score

A probability score is assigned by the model for each word prediction.

6.7 Model Selection

Selection of the model is done based on the highest level of accuracy in predictions, as well as the level of confidence score.

- Model Selection Criteria
- Prediction accuracy
- Value of loss
- Training duration
- Grammar correctness
- Final Model Selection

The output with the highest confidence score is selected as the predicted word. In addition to this, grammar checking is used to validate the choice.

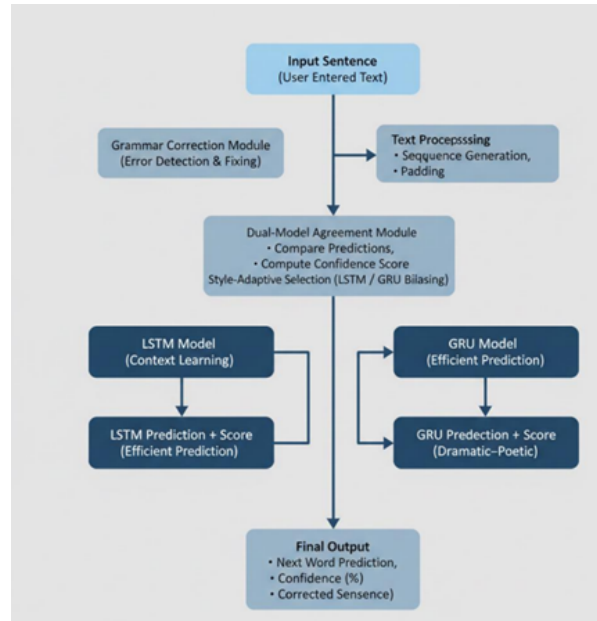


Figure 6.1: Proposed system architecture combining grammar correction, dual-model (LSTM and GRU) prediction, and confidence-based selection mechanism

Chapter 7

Model Architecture

7.1 LSTM Architecture

Long Short-Term Memory Networks are unique types of recurrent neural networks used to solve problems associated with conventional RNNs, like the problem of vanishing gradients. An LSTM network can learn how to predict data sequences by analyzing long-term dependencies; hence, it is quite efficient for predicting the next word in a sequence.

The LSTM network contains a memory cell and three critical gates to control the flow of data into and out of memory.

Input gate Forget gate Output gate

7.1.1 Forget Gate

The forget gate decides which information from the previous memory should be removed. This is important because not all past information is relevant for future predictions. This gate helps the model eliminate unnecessary or outdated information.

7.1.2 Output Gate

The output gate determines what part of the memory should be used to produce the current output.

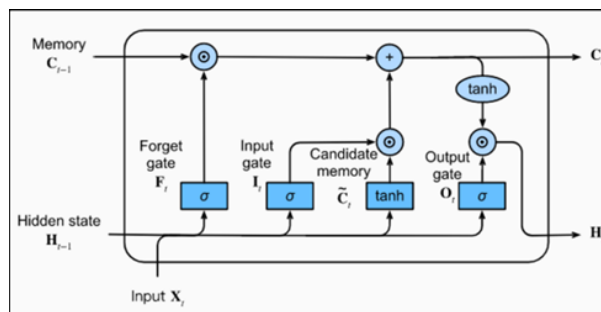


Figure 7.1: Architecture of Long Short-Term Memory (LSTM) network showing forget gate, input gate, output gate, and memory cell operations

- Forget Gate (f) - decides what to remove
- Input Gate (i) - decides what to add
- Cell State (C) - long-term memory
- Output Gate (o) - controls output

The LSTM model processes the input sequence through an embedding layer followed by an LSTM layer that captures contextual relationships between words. The hidden state is updated at each step, and the output is passed through a dense layer to generate the next word prediction.

7.2 GRU Architecture

Gated Recurrent Unit (GRU) is a simplified version of the LSTM architecture. It is designed to achieve similar performance while reducing computational complexity and training time.

Unlike LSTM, GRU uses only two gates:

Update Gate Reset Gate

GRU combines the memory cell and hidden state into a single structure, making it more efficient.

7.2.1 Update Gate

The update gate determines how much of the previous information should be carried forward to the current state.

7.2.2 Reset Gate

The reset gate determines how much past information should be ignored when computing the new hidden state.

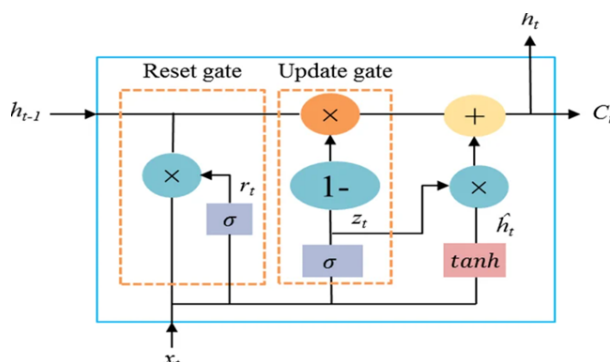


Figure 7.2: Architecture of Gated Recurrent Unit (GRU) showing reset gate, update gate, and hidden state computation

- Update Gate (z) - controls memory update
- Reset Gate (r) - controls forgetting

The GRU model follows a similar structure but uses a simplified architecture for faster computation. It processes the input sequence, updates the hidden state efficiently, and generates the next word prediction through a dense layer.

Chapter 8

Results and Analysis

8.1 Accuracy Comparison

Table 8.1: Accuracy Comparison

Model	Accuracy
LSTM	0.142
GRU	0.192

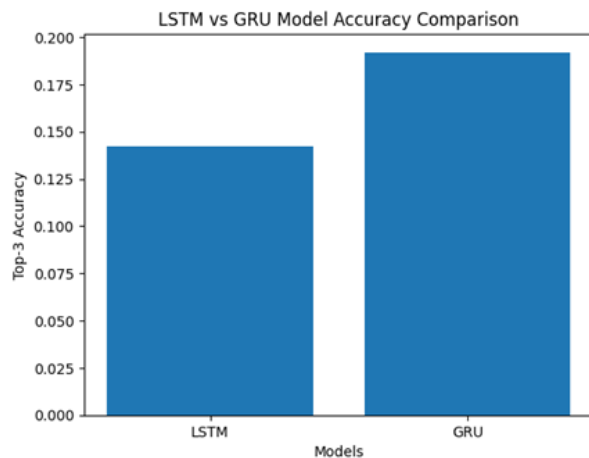


Figure 8.1: Top-3 accuracy comparison between LSTM and GRU models

From the above table, it is observed that the GRU model achieves a higher Top-3 accuracy compared to the LSTM model. This indicates that GRU is more effective in predicting the correct word within the top three suggestions. The improved performance of GRU can be attributed to its simpler architecture and efficient handling of sequential data.

8.2 Confidence Comparison

Table 8.2: Confidence Comparison

Model	Confidence
LSTM	0.063
GRU	0.093

The confidence score represents the probability assigned by the model to its predicted word. The GRU model shows a higher average confidence score than the LSTM model, indicating that it is more certain about its predictions. This suggests that GRU produces more reliable and stable outputs.

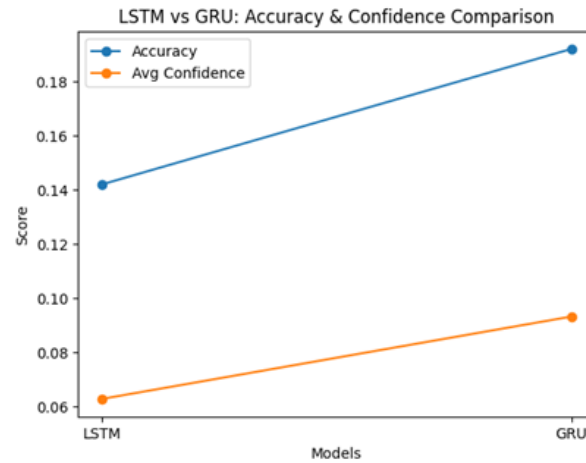


Figure 8.2: Comparison of LSTM and GRU models based on accuracy and average confidence scores

8.3 Observations

- GRU outperforms LSTM in both accuracy and confidence
- GRU requires fewer parameters and trains faster
- LSTM is more complex and computationally intensive
- GRU is more suitable for real-time applications

Chapter 9

Conclusion

In this project, an intelligent system was successfully developed to correct grammatical errors in sentences and provide intelligent word predictions using deep learning approaches. The intelligent system was composed of a rule-based grammatical correction module and sequence prediction approaches such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). The major goal of this project was to enhance the quality of input text by correcting errors in sentences while providing effective word predictions. The grammatical correction module was significant in enhancing the quality of input text by correcting grammatical errors such as subject-verb agreement and incorrect sentence structures. Even though there were some limitations in rule-based approaches, it was an effective preprocessing module in enhancing the quality of input text. The major contribution of this project is based on the implementation and comparison of the LSTM and GRU models for next word prediction. Both models are effective in dealing with sequence data and learning contextual relationships between words. However, based on the experimental results, it is clear that the GRU model is better than the LSTM model in terms of accuracy and efficiency. The GRU has higher top-3 accuracy and is more confident in its predictions. The comparative analysis performed in this study provides valuable insights into the strengths and weaknesses of LSTM and GRU models. In conclusion, the project has successfully fulfilled the set goals in the development of an efficient and functional NLP system for grammar correction and word prediction. The results have also demonstrated that GRU is the best choice in comparison with the LSTM model. This project has the potential to be used as the basis for further research and development in the development of advanced NLP systems.

References

- [1] Hochreiter, S. and Schmidhuber, J., 1997. Long short-term memory. *Neural Computation*, 9(8), pp.1735-1780. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735)
- [2] Mikolov, T., Karafiát, M., Burget, L., Černocký, J. and Khudanpur, S., 2010. Recurrent neural network based language model. In: *INTERSPEECH*, pp.1045-1048. DOI: [10.21437/Interspeech.2010-343](https://doi.org/10.21437/Interspeech.2010-343)
- [3] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y., 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*. DOI: [10.48550/arXiv.1406.1078](https://doi.org/10.48550/arXiv.1406.1078)
- [4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. DOI: [10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762)
- [5] Goldberg, Y., 2017. *Neural network methods for natural language processing*. Morgan & Claypool Publishers. DOI: [10.2200/S00762ED1V01Y201703HLT037](https://doi.org/10.2200/S00762ED1V01Y201703HLT037)
- [6] Brownlee, J., 2019. *Deep learning for natural language processing: Develop deep learning models for your natural language problems*. Machine Learning Mastery.
- [7] Young, T., Hazarika, D., Poria, S. and Cambria, E., 2018. Recent trends in deep learning based natural language processing. *IEEE Computational Intelligence Magazine*, 13(3), pp.55-75. DOI: [10.1109/MCI.2018.2840738](https://doi.org/10.1109/MCI.2018.2840738)
- [8] Jurafsky, D. and Martin, J.H., 2021. *Speech and language processing*. 3rd ed. Draft.
- [9] Pennington, J., Socher, R. and Manning, C.D., 2014. GloVe: Global vectors for word representation. In: *EMNLP*, pp.1532-1543. DOI: [10.3115/v1/D14-1162](https://doi.org/10.3115/v1/D14-1162)
- [10] Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *NAACL-HLT*. DOI: [10.48550/arXiv.1810.04805](https://doi.org/10.48550/arXiv.1810.04805)
- [11] Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I., 2018. Improving language understanding by generative pre-training. OpenAI.
- [12] Radford, A. et al., 2019. Language models are unsupervised multitask learners. OpenAI.
- [13] Sutskever, I., Vinyals, O. and Le, Q.V., 2014. Sequence to sequence learning with neural networks. In: *NeurIPS*. DOI: [10.48550/arXiv.1409.3215](https://doi.org/10.48550/arXiv.1409.3215)
- [14] Kingma, D.P. and Ba, J., 2015. Adam: A method for stochastic optimization. In: *ICLR*. DOI: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980)
- [15] Zaremba, W., Sutskever, I. and Vinyals, O., 2014. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*. DOI: [10.48550/arXiv.1409.2329](https://doi.org/10.48550/arXiv.1409.2329)